

Task No: 4.1

Date: 20/8/25

Aim: Record a ceteria's snack sales for 7 days using a list, compute total and average sales, find the best/worst day, and count how many days crossed a target.

Algorithm:

1. Start
2. Create an empty list `sales = []` ~~to the list using append~~
3. for 7 days, append integer sales to the list using `append()`
4. compute $total = \text{sum}(sales)$ and $avg = total/7$.
5. find $max-val = \max(sales)$, $min-val = \min(sales)$
6. Find corresponding days with `index()`. (add +1 to convert to day number)
7. Count days above a target using `count()` a boolean re-map or with a loop.
8. Stop

Program:

```
# list scenario
```

```
days = 7
```

```
sales = []
```

```
target = 500 # target sales for the day.
```

```
for s in range(8):
```

```
    sample-entries = int(input("Enter the seven days sales count"))
```

```
    sales.append(sample-entries) # list.append()
```

```
total = sum(sales)
```

```
avg = total/days
```

```
max-val = max(sales)
```

```
min-val = min(sales)
```

```
best-day = sales.index(max-val) + 1
```

```
worst-day = sales.index(min-val) + 1
```

```
Print ("Sales (mon - sun):", sales)
```

```
Print ("Total:", total)
```

```
Print ("Average:", round(avg, 1))
```

```
Print ("Best day:", best-day, "with", max-value).
```

```
Print ("Worst day:", worst-day, "with", min-val).
```


Sample Input/output:

enter the seven days sales count: 100

enter the seven days sales count: 450

enter the seven days sales count: 1250

enter the seven days sales count: 580

enter the seven days sales count: 98

enter the seven days sales count: 348

enter the seven days sales count: 900

enter the seven days sales count: 239

Sales (mon --- sun): [100, 450, 1250, 580, 98, 348, 900, 239]

Total: 3974

Average: 567.71

Best Day: 3 with 1250

Worst day: 5 with 98.



Task No: 4.2
Date: 20/8/25

Tuple - Lab Time table.

Aim: To manage and query an immutable daily lab slot schedule using a tuple, demonstrating membership checks, `count()`, `index()` and slicing.

Algorithm

1. Start
2. Define slots as a fixed tuple of integers.
3. Read query hour.
4. Check existence with query in slots.
5. Use `count()`; if positive, use `index()` to find the first position.
6. Slice into morning and afternoon.
7. Print results.
8. Stop.

Python Program

TUPLE scenario.

slots = (9, 11, 14, 16, 14) # immutable daily schedule.

query = 14

exists = (query in slots)

freq = slots.count(query) # tuple.count()

first_pos = slots.index(query) + 1 if exists else "N/A" # tuple.index()

morning = slots[:2]

afternoon = slots[2:]

print("All lab slots:", slots)

print(f"Is {query}:00 present?", exists)

print(f"{query}:00 occurs", freq, "time(s)")

print("First occurrence position (1-based):", first_pos)

print("morning slots:", morning)

print("Afternoon slots:", afternoon).

Sample Input/output:-

All lab slots: (9, 11, 14, 16, 14)

Is 14:00 present? True

14:00 occurs 1 times(s)

First occurrence position (1-based): 3

Morning slots: (9, 11)

Afternoon slots: (14, 16, 14)

Task No: 4.3

Date: 20/08/25

Dictionary - Book Store billing

Aim: To manage a live price list and bill a customer using dictionary methods and view.

Algorithm:

1. start
2. create an empty dictionary prices.
3. Ask the user for number of items in the price list (n1)
4. Repeat for each item:
5. Get the item name
6. Get the item price.
7. Add the item and price to prices.
8. Ask the user for an item to update (or press Enter to skip).
9. If the item exists in prices, get the new price and update it.
10. find the costliest item by checking each item's price.
11. Ask the user for an item to remove (or press Enter to skip)
12. If given, remove that item from prices.
13. show all available items, their prices. the costliest item, and the removed item's price.
14. stop.

Program:

price = {}

n1 = int(input("Enter number of items in price list:"))

for _ in range(n1):

item = input("Enter item name: ")

price = float(input("Enter price of {item}: "))

prices[item] = price.

optional price revision

rev-item = input("Enter item to update price (or press enter to skip): ")

If rev-item in prices:

new-price = float(input(f"Enter new price for {rev-item}: "))

output:

Enter number of items in price list: 3

Enter item name: box

Enter price of box: 15

Enter item name: pen

Enter price of pen: 10

Enter item name: pencil

Enter price of pencil: 5

Enter item to update prices (or enter to skip): box

Enter new price for box: 20

Enter an item from price list (or enter to skip): pen

Available items = ['box', 'pencil']

Prices = [20.0, 5.0]

Costliest item: box at 20.0

Removed 'pen' price (if existed): 10.0


```

prices = update ({ rev-item: new-price }) # dict.update ()
# find costliest item.
costliest-item = None
max-price = 0
for item, price in prices.items():
    if price > max-price:
        max-price = price
        costliest-item = item
# Remove out-of-stock item
remove-item = input("Enter an item an Remove from price list (or press  
Enter to skip):")
removed-price = None
if remove-item:
    removed-price = prices.pop(remove-item, None) # dict.pop()
# Display results.
print("\n Available items:", list(price.keys())) # dict.keys()
print("prices:", list(prices.values())) # dict.values()
if costliest-item:
    print("costliest item:", costliest-item, "at", max-price)
if remove-item:
    print(f"Removed '{remove-item}' price (if existed):", removed-price)

```

VEL TECH	
EX NO.	4
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	
TOTAL (20)	
SIGN WITH DATE	15

Result:

The programs using lists, tuples, Dictionaries are successfully executed.